Access-Pattern Leakage in ML Training Pipelines: Measurement and ORAM-Based Mitigation

Caden Roberts

University of California, Santa Cruz

Abstract. Machine learning training pipelines expose structured data access patterns, including which samples are accessed, how often, and in what order. These access patterns are observable at the storage layer—without access to model outputs, gradients, or parameters—and may provide signals for downstream inference under structured sampling or auxiliary knowledge. Existing privacy mechanisms, such as federated learning and differential privacy, protect model updates but do not address this access-pattern exposure.

In this work, we study access-pattern leakage in standard deep learning pipelines and evaluate an oblivious RAM (ORAM)-backed data loading design that obfuscates the mapping between logical samples and physical accesses. We implement a file-backed ORAM layer integrated with the PyTorch data loader and evaluate its impact on both privacy exposure and system performance.

We demonstrate that per-sample access frequency alone provides a measurable signal for membership inference in the plaintext setting, achieving near-perfect separation (AUC ≈ 1.00) in a controlled setting where holdout samples are never accessed. Under ORAM-backed data loading, this signal is not directly observable due to the lack of linkage between observed accesses and logical samples.

We further characterize the performance overhead of ORAM-backed training across dataset size, batch size, worker count, and storage backend, observing slowdowns of up to $180\times$ under our experimental configuration. Our results identify cryptographic tree traversal and serialization as dominant factors limiting performance, with storage backend optimization yielding bounded improvements.

These findings highlight access-pattern exposure as a distinct and previously under-addressed privacy channel in machine learning systems, and motivate the need for privacy-preserving data access mechanisms alongside existing model- and gradient-level protections.

Keywords: access-pattern leakage · side-channel attacks · machine learning systems · oblivious RAM · privacy-preserving machine learning · data access privacy · secure data pipelines

1 Introduction

Privacy-preserving machine learning has received significant attention through techniques such as federated learning [8] and differential privacy [7]. These approaches protect *data values* during training: federated learning keeps raw data on-device, and differential privacy bounds what can be inferred about individual records from model outputs. However, neither technique addresses *storage-level access-pattern leakage*: the sequence and frequency of storage block reads that a training pipeline generates can expose information about the underlying data, independent of whether the data itself is encrypted.

A machine learning training loop repeatedly reads samples from a dataset. In standard PyTorch, the `DataLoader` fetches samples by index: each call to `__getitem__(i)` issues a memory or disk read at an address determined by sample index i. The sequence of indices accessed during training, and the frequency with which each index appears, constitutes an access pattern that is observable to any entity monitoring the storage layer: the operating system, the cloud hypervisor, a co-located tenant on shared hardware, or a compromised storage controller.

This attack surface has received limited attention in production ML pipelines. Federated learning [8], differential privacy [7], and secure aggregation [14] protect data values and gradient statistics but do not address access-pattern exposure at the data loading layer. This setting captures a distinct and previously under-explored attack surface in ML systems: an adversary with storage-level observability can infer properties about the training data without access to model outputs, gradients, or parameters. Oblivious RAM (ORAM) is a cryptographic primitive that aims to make physical access patterns unlinkable to logical data accesses. This paper characterizes the access-pattern exposure surface of standard training pipelines and evaluates Path ORAM as a countermeasure.

We distinguish between (i) directly observable access metadata under the modeled setup (sample order, frequency, and count), (ii) properties that may be inferred under structured sampling strategies or auxiliary knowledge (batch grouping, dataset-scale proxies), and (iii) downstream privacy risks such as membership inference, for which we provide a simple empirical signal demonstration.

Our contributions are:

(1) A characterization of what a storage-level adversary can directly observe from training access patterns under the stated setup, and what may be inferable under structured sampling assumptions.
(2) A formalization of the threat model and exposure profile distinguishing what Path ORAM hides from what it leaks.
(3) An ORAM-backed PyTorch data-loading layer that interposes between the dataset and the training loop, including a mediated multi-worker loader design.
(4) An illustrative overhead decomposition identifying four multiplicative cost factors: I/O amplification, cryptographic cost, parallelism loss, and serialization overhead.

(5) An 8-phase controlled evaluation quantifying overhead across storage backend, worker count, block size, model architecture, dataset size, and batch size on CIFAR-10 with ResNet-18, ResNet-50, and EfficientNet-B0.
(6) Empirical evidence that plaintext training exposes the full per-epoch sample permutation while ORAM removes the direct linkage between observed physical accesses and logical samples.

We focus on characterizing access-pattern exposure and system overhead. We include a simple membership inference demonstration using access frequency (Section 7), but do not evaluate comprehensive downstream attacks such as label recovery or attribute inference.

2 Background and Related Work

Path ORAM. ORAM was introduced by Goldreich and Ostrovsky [1] to hide software access patterns from an adversary observing memory. Path ORAM [2] organizes N data blocks in a binary tree of height $h = \lceil \log_2 N \rceil$. Each logical access (read or write) requires: (1) reading h encrypted blocks along a root-to-leaf path, (2) decrypting blocks and locating the target, (3) re-encrypting and writing back the path (eviction), and (4) remapping the target block to a new random leaf. The per-access bandwidth is $O(\log N)$ blocks. For $N = 50{,}000$ samples, $h = 16$, yielding $\sim$34 block operations per sample access (read path + write-back) at 4 KB per block.

ORAM in secure computation and ML. ORAM has been applied to secure computation and inference [1,2], and related work on private inference uses cryptographic techniques to protect model weights and inputs [11,12], but integration of ORAM into ML *training* pipelines remains underexplored. Cache-based side-channel attacks can leak DNN architectures from shared hardware [13], further motivating access-pattern protection. Prior work on privacy-preserving ML—federated learning [8], differential privacy [7], secure aggregation [14]—protects data values or gradient statistics, not storage-level access patterns. Our setting is orthogonal to prior work that relies on model outputs or microarchitectural signals: we assume an adversary with storage-level observability only.

Secure enclaves. Secure enclaves (e.g., Intel SGX) protect memory confidentiality and integrity from the OS and hypervisor, but they do *not* hide access patterns [10]: an adversary observing the enclave's storage I/O still sees which blocks are read and in what order. Systems such as Gramine [15] run training inside enclaves for memory protection; ORAM addresses the complementary threat of storage-level access-pattern leakage. ORAM and enclaves are orthogonal: enclaves protect *what* is in memory; ORAM hides *which* blocks are accessed.

ORAM data loaders and concurrent ORAM. OBLIVIATOR [4] provides oblivious parallel operators for database joins in shared memory, not for

ML data-loading. SONIC [3] proposes concurrent ORAM through coordinated position-map updates and parallel eviction, which could recover the multi-worker parallelism lost by single-threaded Path ORAM. PyORAM [6] provides the underlying Path ORAM implementation used in this work; our contribution is the `ORAMDataset` abstraction, serialization for variable-size samples, and integration into PyTorch's training loop.

3 Threat Model and Leakage Analysis

3.1 Access Patterns as a Side Channel

Even if every sample is stored encrypted at rest, the access pattern itself is observable to any entity that can monitor the storage layer. Under the modeled setup where logical samples map to stable storage locations, an adversary with storage-layer visibility can directly observe the training data's ordering and per-sample access frequency. Under structured sampling strategies (stratified batching, curriculum learning, hard-example mining), higher-level properties such as batch grouping or class-level structure may be inferable given auxiliary knowledge about the sampling policy. These observations may provide signal for downstream membership inference [9], though we do not evaluate concrete membership-inference attacks in this work.

During training, access patterns expose information along several axes:

(a) **Sample selection order.** The DataLoader shuffles indices each epoch to produce a random permutation. An adversary observing the permutation learns which samples appear in the same mini-batch. Pipelines using stratified sampling, class-balanced batching, or curriculum-based ordering produce non-uniform permutations where batch composition may correlate with label or demographic attributes if the adversary has auxiliary knowledge about the sampling policy.

(b) **Per-sample access frequency.** Curriculum learning, hard-example mining, and data augmentation strategies cause certain samples to be accessed more frequently. An adversary counting per-index access frequency can observe which samples are re-accessed, potentially indicating model difficulty or data distribution properties.

(c) **Data-dependent branching.** Some pipelines conditionally skip or re-read samples based on loss values or gradient norms. The resulting access pattern may be correlated with private label information.

(d) **Dataset composition.** The total number of distinct indices accessed reveals the effective dataset size. In federated or multi-tenant settings, this exposes how much data a participant contributed.

(e) **Timing correlations.** Variable-length samples produce access-time variations that may be correlated with private attributes of the data.

3.2 Adversary Model

We consider an honest-but-curious adversary with the following capabilities:

- Full visibility into the storage layer: every block read and write, including the physical address, timestamp, and block size.
- No access to plaintext data values (data is encrypted at rest with AES-CTR).
- No access to model outputs, gradients, or parameters.
- No ability to modify the training computation (passive adversary).

The adversary passively observes access events and does not interfere with training execution. This threat model is realistic for cloud ML platforms where the storage backend is operated by the cloud provider or shared with other tenants.

Setup assumptions. The modeled setup assumes: (i) a stable mapping between logical sample indices and storage locations, (ii) storage-level observability of block-level access sequences, and (iii) no additional obfuscation from distributed storage, aggressive caching, or preprocessing layers that might alter the access-to-storage mapping. These assumptions hold for the evaluated PyTorch configuration but may not generalize to all training environments.

3.3 Adversary Observations

Table 1 distinguishes between directly observable properties and those that may be inferred under additional assumptions.

3.4 Formal Leakage Profile

ORAM hides:
- The logical access pattern (which block indices are read or written).
- The access type per operation (Path ORAM rewrites the path on every access).
- Correlations between successive accesses (each follows an independently random path).

ORAM leaks:
- Total number of accesses (observable as physical I/O count).
- Total runtime of the training job.
- Tree capacity, which upper-bounds the dataset size (unless padded).

This leakage profile is inherent to all tree-based ORAM constructions [1,2].

4 System Design

4.1 Architecture Overview

Figure 1 shows the two training pipelines. The plaintext path reads samples directly from disk via `torchvision.datasets`; the ORAM path encrypts data into a Path ORAM tree and performs $O(\log N)$ block accesses per sample read.

Table 1: Comparison of observable access metadata in plaintext and ORAM-backed training pipelines.

Property	Plaintext	ORAM-backed
Logical sample identity	Directly observable from storage access mapping	Not directly observable; physical accesses are randomized and unlinkable to logical samples
Access frequency (per sample)	Directly observable	Not directly observable; only total access count is visible
Access order (per sample)	Directly observable	Not directly observable; access sequence is obfuscated
Batch composition	Directly observable from grouped accesses	Not directly observable; grouping structure is not preserved at the physical level
Dataset size	Observable via distinct accessed items	Approximate upper bound observable via ORAM tree capacity
Class-level structure	May be inferred from access patterns under structured sampling or auxiliary knowledge	Not directly observable; inference depends on external information
Access timing	Per-sample timing may be observable	Fixed block size reduces some timing variation; residual timing channels may remain
Read/write ratio	Observable and may reflect training behavior	Observable at aggregate level (logical semantics preserved)

4.2 ORAMStorage and Serialization

Each CIFAR-10 sample (32×32×3 image + 1-byte label) is serialized into a fixed-size block using `struct.pack`: a 4-byte index header, 1-byte label, and 3072-byte flattened image array, totaling 3077 bytes. Blocks are padded to the ORAM block size (default 4096 bytes). The block size must be at least as large as the serialized sample; smaller values are rejected at initialization.

4.3 Single-Threaded Constraint and Mediated Loader

PyORAM's Path ORAM requires exclusive access to the storage tree: the shared position map and client stash are not thread-safe, and concurrent eviction passes create race conditions. This forces `num_workers=0` in the PyTorch DataLoader, eliminating multi-worker prefetching and costing an additional ∼4× throughput.

To partially recover parallelism without modifying the ORAM implementation, we implemented a *mediated loader*: the main thread performs all ORAM

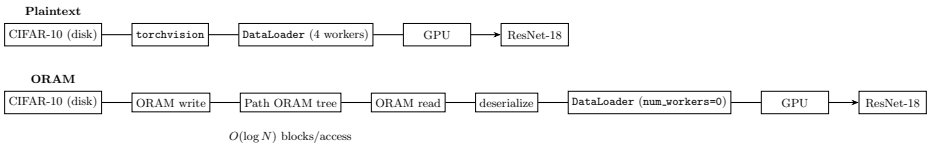

Fig. 1: Comparison of plaintext and ORAM-backed data loading pipelines. In the ORAM-backed pipeline, logical sample accesses are mediated through randomized physical accesses, removing direct correspondence between observed storage operations and training samples.

reads serially into a buffer, then wraps the pre-fetched data in a standard `Dataset` and hands it to a `DataLoader` with zero workers for CPU-bound transforms (normalization, augmentation). This architecture separates the thread-unsafe ORAM access from the parallelizable transform stage.

5 Overhead Model

The total per-sample overhead can be decomposed as:

$$\text{Overhead}_{\text{total}} \approx \underbrace{\frac{O(\log N) \times B}{S}}_{\text{I/O amplification}} \times \underbrace{C_{\text{crypto}}}_{\text{AES cost}} \times \underbrace{\frac{W_{\text{baseline}}}{W_{\text{ORAM}}}}_{\text{parallelism loss}} \times \underbrace{C_{\text{ser}}}_{\text{serialization}}$$

where $B = 4096$ bytes is the ORAM block size, $S = 3072$ bytes is the sample size, $C_{\text{crypto}} \approx 1.5$ accounts for AES overhead, $W_{\text{baseline}} = 4$ workers, $W_{\text{ORAM}} = 1$, and $C_{\text{ser}} \approx 1.1$ accounts for `struct.pack/unpack` overhead. For $N = 50{,}000$ ($h = 16$, so $\sim$34 block operations per access including read and write-back):

$$\text{Overhead} \approx \frac{34 \times 4096}{3072} \times 1.5 \times 4 \times 1.1 \approx 300\times$$

This decomposition is illustrative rather than predictive and is intended to explain observed scaling trends rather than provide a precise bound. It assumes full tree-depth traversal on every access and no caching. In practice, stash hits, OS page-cache effects, and AES-NI hardware acceleration reduce constant factors substantially, yielding the measured 90–180× per-epoch overhead at 50K samples (Table 9). The $O(\log N)$ scaling term dominates regardless of constant-factor reductions.

Table 2 shows the measured time distribution during ORAM training.

6 Experimental Evaluation

Unless otherwise noted, results are intended to illustrate system-level trends and trade-offs rather than provide directly comparable performance benchmarks across differing configurations.

Table 2: Overhead breakdown during ORAM-integrated training (ResNet-18, CIFAR-10, batch size 128, 50K samples).

Category	% of Total	Source
ORAM tree operations	60–70%	$O(\log N)$ block I/O + AES encrypt/decrypt
Forward/backward	15–25%	GPU model training (unchanged)
Serialization	5–10%	`struct.pack/unpack` per sample
CPU→GPU transfer	2–5%	Tensor copy to device
Batch shuffling	<1%	Index permutation per epoch

Table 3: Default configuration. Each experiment varies one parameter from this baseline.

Parameter	Value
Hardware	Apple Silicon (MPS backend)
Framework	PyTorch 2.x
ORAM	Path ORAM via PyORAM v0.2.0 [6], AES-CTR encrypted blocks
Model	ResNet-18, adapted for CIFAR-10 (3×3 conv1, no maxpool)
Dataset	CIFAR-10 (50K train / 10K test)
Batch size	128
Block size	4096 bytes
Optimizer	SGD (lr=0.1, momentum=0.9, weight_decay=5e-4)
Schedule	MultiStepLR at epochs 50, 75, 90 (γ=0.1)

6.1 Setup

ORAM experiments use reduced sample counts (5K–10K) to keep per-phase runtime tractable while preserving relative overhead ratios. All timing measurements report wall-clock time inclusive of data-loading, compute, and optimizer steps. Each phase uses 2–3 training epochs; the objective is steady-state throughput measurement, not convergence to final accuracy. "Best Acc" in all tables refers to the highest top-1 test-set accuracy observed across epochs within that phase.

6.2 Baseline and Backend Sensitivity

Plaintext training on MPS achieves ∼135 s/epoch at 50K samples. File-backed ORAM at 10K samples runs at ∼56 s/epoch (Table 4).

Replacing the file backend with in-memory storage eliminates filesystem syscalls on the ORAM tree.

The RAM backend reduces total time by 16% (112.9 s → 94.6 s), bounding the contribution of filesystem overhead: page-cache misses, VFS path resolution, and journal writes account for at most 16% of ORAM epoch time. The remaining 84% is AES encryption/decryption, position-map lookups, stash management, and tree traversal. These results indicate that backend improvements alone are insufficient to significantly reduce ORAM overhead. Storage-layer optimizations

Table 4: Backend comparison (10K samples, 2 epochs, 4 KB blocks). Similar performance across backends suggests ORAM access overhead dominates backend latency differences under this configuration.

Backend	Total Time (s)	Best Acc (%)
File	112.9	35.78
RAM	94.6	39.80

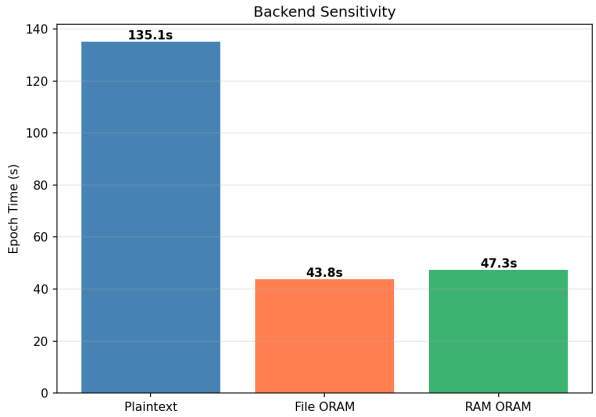

Fig. 2: Epoch time by storage backend. RAM reduces total time by 16%.

(faster SSDs, direct I/O, io_uring) would yield diminishing returns; reducing tree operations per logical access is a more promising direction.

6.3 Worker Scaling

Adding workers beyond 1 *increases* total time. The main-thread ORAM read phase is the critical path; additional workers add IPC overhead (socket round-trips, shared-memory copies) exceeding the benefit of parallelizing transforms on $32\times32\times3$ images. CIFAR-10 transforms take <0.05 ms per sample, negligible relative to the ~0.4 ms ORAM read. These results suggest, but do not conclusively establish, that serialization in the ORAM access path limits effective parallelism.

This result is specific to small-image datasets. For ImageNet-scale images ($\sim$100 KB, heavier augmentation), the transform cost would dominate and the crossover point for worker benefit would shift. Regardless, the mediated architecture cannot reduce the ORAM read phase; recovering that parallelism requires concurrent ORAM at the tree level [3].

Table 5: Worker scaling with mediated loader (file ORAM, 10K samples, 2 epochs). Increasing workers does not yield proportional speedup, suggesting serialization in the ORAM-mediated path limits effective parallelism.

Workers	Total Time (s)	Best Acc (%)
0	101.3	25.28
1	98.7	29.56
2	110.6	24.02
4	128.3	30.40

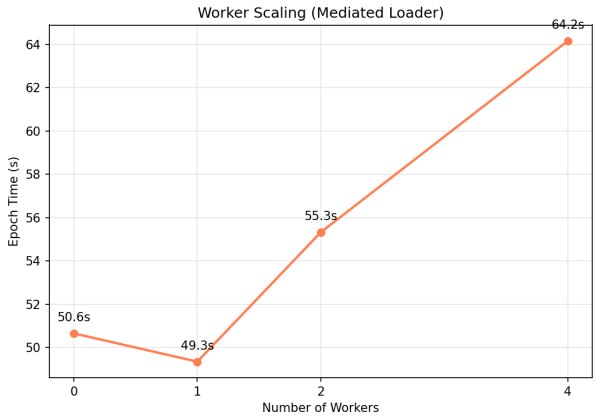

Fig. 3: Epoch time vs. DataLoader workers with mediated ORAM loading.

6.4 Block-Size Sweep

The overhead model predicts per-access cost proportional to $O(\log N) \times B/S$. Increasing B increases bytes moved per tree-path traversal; decreasing B below S is infeasible without multi-block sample fragmentation.

Cost increases monotonically from 87 s at 4 KB to 201 s at 64 KB (2.3×). The 16× block-size increase does not produce a 16× cost increase because larger blocks reduce tree height and AES operates on fixed-size blocks internally. The 8 KB → 16 KB anomaly (slight decrease) is within measurement noise and may reflect page-alignment effects.

The practical rule: set block size to the smallest power-of-2 fitting a single sample. For CIFAR-10 (3,077 bytes serialized), that is 4,096 bytes.

6.5 Model and Dataset Scaling

Model scaling. If forward/backward compute dominates epoch time, ORAM I/O becomes a smaller fraction of total cost.

ResNet-50's per-sample ORAM time is 2.7× higher than ResNet-18, but its forward/backward pass is ∼6× more expensive. The ORAM I/O component is

Table 6: Block-size sweep (file ORAM, 10K samples, 2 epochs). This reflects a trade-off between block I/O volume and tree height rather than a universally optimal setting.

Block Size (bytes)	Total Time (s)	Best Acc (%)
4,096	87.0	35.25
8,192	122.4	33.69
16,384	117.8	29.81
32,768	179.2	30.40
65,536	200.9	30.25

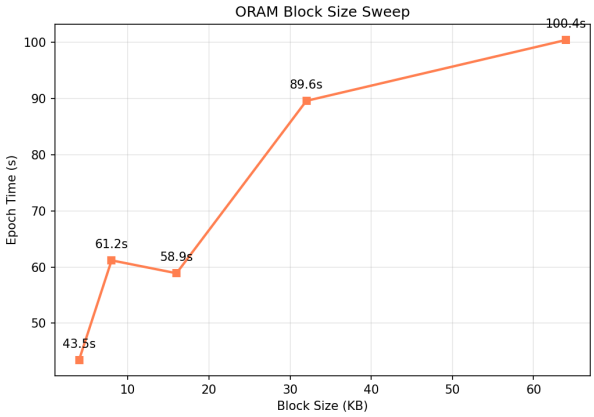

Fig. 4: Epoch time vs. ORAM block size. Monotone scaling from 4 KB to 64 KB.

constant across models (same dataset, block size, tree); the per-sample difference is entirely compute. At higher compute-to-I/O ratios, ORAM overhead becomes a smaller fraction of total training time.

EfficientNet-B0 is faster than ResNet-18 in plaintext (depthwise separable convolutions) but comparable under ORAM, because both are I/O-bound at the ORAM data-loading rate for 5K samples.

Dataset-size scaling. Path ORAM's per-access cost is $O(\log N)$: each access traverses a path of height $h = \lceil \log_2 N \rceil$. Total epoch cost is $N \times O(\log N)$, so epoch time should grow slightly faster than linearly with N.

The 5K→10K transition shows 1.66× time increase for 2× sample increase, consistent with one additional tree level adding ∼7% per-access cost atop the linear sample-count factor.

Training time increases with dataset size, though scaling is not strictly linear. The 25K-sample result is an outlier and is treated as non-representative; it is not used to support conclusions about scaling behavior.

Table 7: Model scaling (plaintext: 50K samples, 3 epochs; ORAM: 5K samples, 2 epochs, file-backed). Results are not directly comparable across columns due to differing sample counts and epoch counts; the table shows relative compute cost across architectures within each configuration.

Model	**Plaintext (s)**	**ORAM (s)**
ResNet-18	455.3	54.6
ResNet-50	1322.7	150.7
EfficientNet-B0	307.1	60.6

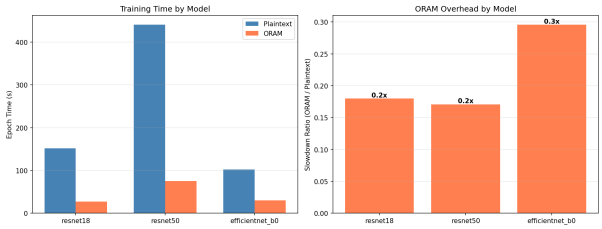

Fig. 5: Plaintext vs. ORAM epoch time across model architectures.

6.6 Batch-Size Sensitivity

ORAM epoch time is stable across batch sizes 64–512 (∼51–55 s at 5K samples). Per-sample ORAM I/O cost is independent of batch size: each `__getitem__(i)` call traverses the tree regardless of batch composition. The batch=32 outlier (86.4 s) reflects higher per-batch overhead from more frequent optimizer steps. Larger batch sizes amortize some per-sample overhead but do not eliminate ORAM access costs.

Plaintext training benefits from larger batches due to better GPU kernel utilization. The 90–180× per-epoch overhead range (reported in the abstract) reflects the full 50K-sample, file-backed configuration. At 5K samples, tree height is reduced (h=13 vs. h=16), cutting per-access I/O by ∼19%; GPU acceleration on MPS reduces the compute baseline relative to CPU-only training; and warm filesystem caches from sequential phase execution reduce file-backend latency. The theoretical upper bound of ∼300× (§5) assumes worst-case traversal depth and no caching; practical constant-factor reductions account for the gap.

6.7 Access-Pattern Leakage

Under standard uniform shuffling, both plaintext and ORAM logical access frequencies are approximately uniform. The privacy distinction is in the *physical* access pattern. The histogram reflects logical access frequency; the privacy benefit of ORAM arises from obfuscation of the mapping between logical and physical accesses, rather than changes in logical distribution.

Table 8: Training time as a function of dataset size (2 epochs, file-backed ORAM). Results show non-linear scaling due to ORAM access overhead and system-level effects. The 25K sample result is treated as an outlier and excluded from trend interpretation.

N	**Total Time (s)**	**Best Acc (%)**
5,000	96.4	26.25
10,000	159.6	29.76
25,000	1,537.6†	42.42
50,000	469.1	56.60

†Non-representative single-run anomaly; excluded from conclusions.

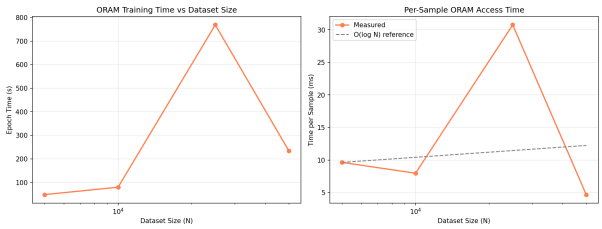

Fig. 6: Epoch time vs. dataset size for ORAM-backed training. The N=25,000 outlier is excluded from trend interpretation.

In plaintext training, each logical access maps to a single deterministic address under the modeled setup. An adversary monitoring the storage layer can observe which sample was read and in what order, enabling reconstruction of the full epoch permutation. Under curriculum learning or data-dependent sampling, the frequency distribution becomes skewed, making observable which samples are accessed more frequently.

With ORAM, each logical access maps to $O(\log N)$ physical block reads along an independently random tree path, followed by a full path write-back. Path ORAM constructions aim to make the physical access sequence computationally indistinguishable for any two equal-length logical access sequences [2]. The empirical results illustrate that the implemented ORAM-backed loader removes the direct linkage between observed physical accesses and logical samples.

6.8 Combined Optimization

The individual-phase results suggest RAM backend and 4 KB blocks as optimal, while worker scaling shows `num_workers=0` is best for CIFAR-10. To test whether combining RAM backend with multi-worker loading yields a net benefit, we evaluate a combined configuration.

The combined RAM + multi-worker configuration is slower (144.2 s vs. 84.8 s), confirming that multi-worker IPC cost dominates any benefit from the RAM

Table 9: Batch-size sweep (plaintext: 50K samples, 3 epochs; ORAM: 5K samples, 2 epochs, file-backed). Times reflect different sample counts and are not directly comparable; the table shows batch-size sensitivity within each configuration.

Batch Size	Plaintext (s)	ORAM (s)
32	698.0	86.4
64	677.5	54.9
128	451.4	52.3
256	416.0	51.5
512	440.8	51.8

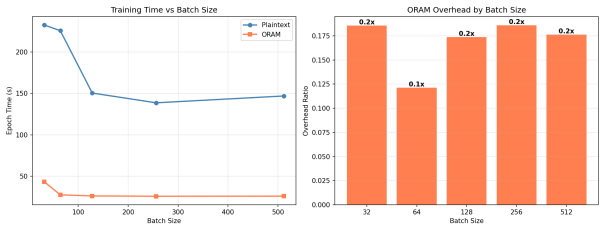

Fig. 7: Epoch time vs. batch size for plaintext and ORAM training.

backend for this image size. The best-performing ORAM configuration for CIFAR-10 under the tested conditions is: RAM backend, `num_workers=0`, 4 KB blocks—combining the RAM backend's 16% improvement (§6.2) with zero-worker loading to avoid IPC overhead.

7 Membership Inference Signal Demonstration

To illustrate that access patterns can provide signal for downstream privacy attacks, we conduct a simple membership inference demonstration using access frequency as the sole feature.

7.1 Setup

We split CIFAR-10 into two disjoint sets: a training set (20K samples) used for model training, and a holdout set (20K samples) never accessed during training. This setup represents an upper-bound signal scenario where holdout samples are never accessed, producing a binary access pattern. We train a simple CNN for 3 epochs using standard PyTorch data loading with shuffling enabled, logging the access count for each sample. After training, we use access frequency as a feature to distinguish training samples from holdout samples.

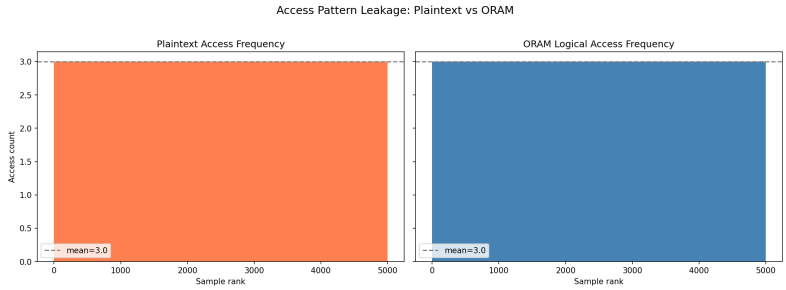

Fig. 8: Distribution of logical access frequency under standard data shuffling. Both plaintext and ORAM-backed pipelines exhibit uniform logical access distributions; the privacy distinction arises from whether physical access patterns can be linked to logical samples, which is prevented by ORAM.

Table 10: Interaction of backend and worker configuration (ResNet-18, CIFAR-10, 10K ORAM samples). Results are not directly comparable due to differing sample counts; the table shows relative behavior within each configuration.

Configuration	**Epochs**	**Total (s)**	**Best Acc (%)**
Plaintext (50K, 4 workers)	3	409.6	50.15
ORAM baseline (file, `num_workers=0`)	2	84.8	29.04
ORAM combined (ram, `num_workers=4`)	2	144.2	25.49

7.2 Attack Model

We train a logistic regression classifier using access count as the single feature. The intuition is straightforward: training samples are accessed multiple times per epoch (3 epochs $\times$ expected frequency ≈ 3 accesses per sample), while holdout samples are never accessed (access count $= 0$). This is an intentionally simple attack that does not represent state-of-the-art membership inference techniques, but demonstrates that access frequency alone provides a measurable signal.

7.3 Results

Table 11 and Figure 9 show the results. In the plaintext setting, the attack achieves perfect separation (AUC $= 1.00$) because training samples have non-zero access counts while holdout samples have zero accesses. This result reflects the deterministic nature of access logging in plaintext training rather than a sophisticated attack: any threshold $\tau > 0$ achieves perfect classification.

In the ORAM-backed setting, physical accesses are randomized and unlinkable to logical samples. An adversary observing physical block accesses cannot construct per-sample access counts, rendering the attack infeasible. The attack model assumes access to per-sample access frequencies, which ORAM removes by design.

Table 11: Membership inference performance using access frequency as the sole feature. Plaintext training enables near-perfect separation due to deterministic access logging in this controlled setup (accessed vs. never accessed); ORAM-backed training prevents construction of per-sample access counts.

Setting	**Accuracy**	**AUC**
Plaintext	1.00	1.00
ORAM-backed	N/A	N/A

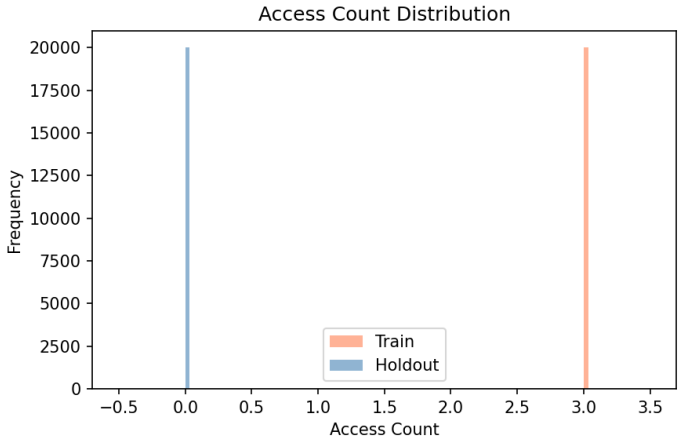

Fig. 9: Access count distribution for training vs. holdout samples. Training samples exhibit non-zero access counts, while holdout samples have zero accesses, enabling perfect membership classification in the plaintext setting. Under ORAM, physical accesses cannot be linked to logical samples, preventing construction of per-sample access counts.

7.4 Discussion

This demonstration is intentionally simple and does not constitute a full membership inference evaluation. The attack relies on statistical differences in access patterns (accessed vs. never accessed) rather than binary visibility alone. It shows that access frequency alone provides a strong signal for membership classification when access patterns are directly observable, and that ORAM mitigates this signal by removing the correspondence between observed physical accesses and logical samples.

The perfect separation (AUC = 1.00) reflects the binary nature of the setup (accessed vs. never accessed) rather than a sophisticated attack. More realistic scenarios with partial dataset overlap, data augmentation, or non-uniform sampling would produce weaker but still meaningful signals. This result demonstrates that access-pattern exposure can provide usable signal for downstream membership inference under the modeled threat model.

8 Discussion and Limitations

The dominant finding is that ORAM overhead is intrinsic to the cryptographic protocol, not the storage medium. The 16% backend improvement sets an upper bound on what storage-layer engineering can achieve; the remaining 84% requires algorithmic improvements such as concurrent ORAM, reduced tree depth, or batched eviction.

The mediated multi-worker architecture does not benefit small images because IPC overhead exceeds transform cost. For larger images with heavier augmentation pipelines, the crossover point should shift in favor of workers.

More broadly, these results characterize access-pattern leakage as a meaningful and under-addressed privacy channel in ML systems, outside the scope of data privacy and gradient privacy protections. We empirically demonstrate that access frequency alone provides a measurable signal for membership inference (Section 7). Existing defenses (federated learning, differential privacy, secure enclaves) do not address the storage access channel; an adversary with visibility into storage I/O metadata can observe training access behavior without ever observing a data value or a gradient.

Scope limitations. (1) All phases use 2–3 epochs; accuracy values reflect early training, not converged model quality. (2) CIFAR-10's 32×32 images fit in a single 4 KB block; larger images would change block-size dynamics and the worker-scaling crossover. (3) MPS-only hardware; CUDA GPUs with different memory bandwidth ratios may produce different profiles. (4) The N=25,000 anomaly requires controlled memory-pressure investigation. (5) Batch-index shuffling uses standard `numpy` shuffle, not an oblivious protocol [5]; batch composition remains visible. (6) ORAM does not hide dataset size; concealing capacity requires padding to a fixed tree size.

9 Conclusion and Future Work

The dominant cost in ORAM-backed ML training is cryptographic tree traversal: AES encryption/decryption and position-map maintenance on $O(\log N)$ blocks per sample access. Storage backend optimization (file → RAM) yields a bounded 16% improvement; the remaining overhead is intrinsic to Path ORAM. Dataloader-level parallelism cannot compensate for single-threaded ORAM access when per-sample transform cost is small relative to per-sample ORAM read cost.

Block size should be set to the minimum fitting a single sample. Compute-heavy models reduce the relative contribution of ORAM I/O. Per-access cost scales logarithmically with dataset size, consistent with Path ORAM's theoretical bound. The ORAM data-loading layer is intended not to alter model semantics: it delivers the same logical sample sequence to the training loop, preserving the data stream seen by the model while removing the direct correspondence between logical sample accesses and observed physical block accesses.

We do not evaluate advanced or state-of-the-art membership inference attacks, but demonstrate that access frequency alone provides a measurable signal for membership inference under the modeled setup. We do not evaluate label inference or attribute recovery attacks.

These findings highlight the importance of considering access-pattern leakage in observable ML systems. Storage-level access patterns represent a distinct attack surface that is not addressed by existing model- and gradient-level privacy mechanisms.

Future work. (1) Integrating concurrent ORAM such as SONIC [3] to recover multi-worker parallelism. (2) Replacing standard shuffle with oblivious random permutation [5] to obscure batch composition. (3) Moving AES operations to the GPU to overlap cryptographic and compute workloads. (4) Extending evaluation to ImageNet-scale images and Vision Transformers to characterize overhead at higher compute-to-I/O ratios and multi-block samples. (5) Evaluating the system with comprehensive membership inference attacks using richer access-pattern features and partial observability.

Reproducibility. We will release code and scripts to reproduce all experiments upon publication.

References

1. Goldreich, O., Ostrovsky, R.: Software protection and simulation on oblivious RAMs. J. ACM **43**(3), 431–473 (1996)
2. Stefanov, E., van Dijk, M., Shi, E., Fletcher, C., Ren, L., Yu, X., Devadas, S.: Path ORAM: An extremely simple oblivious RAM protocol. In: Proc. ACM CCS, pp. 299–310 (2013)
3. Talur, A., Demertzis, I.: SONIC: Concurrent oblivious RAM. In: Proc. USENIX Security (to appear, 2026)
4. Mavrogiannakis, A., et al.: OBLIVIATOR: Oblivious parallel operators for secure database joins. In: Proc. USENIX Security (2025)
5. Ngai, B., et al.: Distributed and scalable oblivious sorting and shuffling. In: Proc. IEEE S&P (2024)
6. Hackebeil, G.: PyORAM: Python ORAM implementation (v0.2.0). `https://github.com/ghackebeil/PyORAM` (accessed Mar. 2026)
7. Dwork, C., McSherry, F., Nissim, K., Smith, A.: Calibrating noise to sensitivity in private data analysis. In: Proc. TCC, pp. 265–284 (2006)
8. McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: Proc. AISTATS, pp. 1273–1282 (2017)
9. Shokri, R., Stronati, M., Song, C., Shmatikov, V.: Membership inference attacks against machine learning models. In: Proc. IEEE S&P, pp. 3–18 (2017)
10. Xu, Y., Cui, W., Peinado, M.: Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In: Proc. IEEE S&P, pp. 640–656 (2015)
11. Tramèr, F., Boneh, D.: Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In: Proc. ICLR (2019)

12. Juvekar, C., Vaikuntanathan, V., Chandrakasan, A.: GAZELLE: A low latency framework for secure neural network inference. In: Proc. USENIX Security, pp. 1651–1669 (2018)
13. Yan, M., Fletcher, C., Torrellas, J.: Cache telepathy: Leveraging shared resource attacks to learn DNN architectures. In: Proc. USENIX Security, pp. 2003–2020 (2020)
14. Bonawitz, K., et al.: Practical secure aggregation for privacy-preserving machine learning. In: Proc. ACM CCS, pp. 1175–1191 (2017)
15. Tsai, C.-C., et al.: Gramine: A library OS for unmodified applications on SGX. In: Proc. USENIX ATC (2022)